Identifying Interaction Design and Functional Layout

Practical implementation of interaction design and functional layouts

Case study: **Personal App — Notes Module** (Rich Text Notes with Search, Color Coding, Priority, Trash)

Submitted by: SAI CHANDAN KS (23MIS0115)

Tools: Flutter 3.22+ · Dart 3.7+ · Provider · SQLite

1. Aim

To identify user requirements for a notes application, design an interaction prototype based on those requirements, and implement it with consistent placement, wording, and colour so the user can build an accurate mental model of the interface — demonstrated through the **Notes module** of Personal App, a local-first rich text notes application with search, colour coding, priority tagging, trash management, and reminder scheduling.

2. Objectives

1. Understand the principles of interaction design and functional layout.
2. Translate user requirements into a working design prototype (Activity 1).
3. Apply consistency rules — button placement, label wording, colour scheme — across every screen (Activity 2).
4. Evaluate whether the resulting interface supports an accurate, predictable mental model.

3. Theory

3.1 Interaction Design

Interaction design (IxD) is the practice of designing how a user and a system communicate — what happens when the user clicks, types, taps, or navigates. It focuses on the sequence of screens and states a user moves through to complete a task, not just the visual appearance of a single screen. In the Notes module, this includes the transition from the notes list to the editor, the flow from an

empty state to a populated list, and the feedback loops (snackbars, progress indicators, confirmation dialogs) that confirm user actions.

3.2 Functional Layout

A functional layout arranges elements on a screen — navigation, primary actions, forms, content — so that the arrangement itself supports the task. In the Notes module, the primary action (create a new note) is always a floating action button at the bottom-right, the search bar is always at the top, and the bottom navigation bar persists with the Notes tab highlighted.

3.3 Consistency Principles

Three concrete rules are applied throughout the Notes module:

- **Placement consistency** — The FAB (create) is always bottom-right. The trash icon is always top-right in the editor. The search bar is always the first element in the app bar.
- **Wording consistency** — “Save” commits edits, “Cancel” discards, “Delete” moves to trash, “Discard” confirms loss of unsaved changes. These labels never change.
- **Colour consistency** — Blue = primary action, red = destructive/delete, amber = warning/medium priority, green = success/low priority. Each colour keeps one meaning.

3.4 Mental Model

A mental model is the user’s internal expectation of how a system behaves, built from repeated exposure to consistent patterns. When placement, wording, and colour stay consistent, a user who learns the interface on the Notes List screen can correctly predict how the Editor, Trash, Color Picker, and Priority Picker will behave — reducing errors and learning time.

4. Software / Tools Used

Tool	Purpose
Flutter 3.22+	Cross-platform UI framework (Material 3 design system)
Dart 3.7+	Programming language with sound null safety
Provider	State management (ChangeNotifier-based)
SQLite (sqflite)	Local encrypted relational database
flutter_quill	Rich text editor (bold, italic, lists, headers, links, images)

Tool	Purpose
flutter_local_notifications + timezone	Push notification scheduling for note reminders
shared_preferences	Key-value storage for preferences
share_plus	Share note content to other apps
intl	Date/time formatting

5. Activity 1 — Design and Prototype the Interaction

Case study: **Personal App — Notes Module**.

5.1 Step 1 — Identify user requirements

#	User requirement	Screen / interaction it maps to
1	The user needs to create and organize rich-text notes with formatting.	Notes Editor — Quill rich text with bold, italic, lists, headers, links, images
2	The user needs to visually distinguish notes by colour.	Color Picker — 14 presets + custom HSV/Hex input
3	The user needs to set priority levels on notes.	Priority Picker — None / Low (green) / Medium (amber) / High (red)
4	The user needs to find notes quickly without scrolling.	Search bar — real-time filtering by title and content
5	The user needs to bulk-delete or bulk-move notes to trash.	Selection Mode — long-press to enter, select-all, bulk delete
6	The user needs to recover accidentally deleted notes.	Trash bottom sheet — restore or permanently delete per item
7	The user needs to pin or favourite important notes.	Note card — pin icon on list + favourite star in editor
8	The user needs to set reminders on notes.	Editor — reminder time picker with notification scheduling
9	The user needs to see when a note was last updated.	Note card — relative timestamp (“2h ago”, “Yesterday”)

#	User requirement	Screen / interaction it maps to
10	The user must confirm before losing unsaved changes.	Discard Changes dialog — “Cancel” vs “Discard”
11	The user needs feedback when notes cannot be loaded.	Loading spinner + Error state with Retry button
12	The user needs a clear empty state when no notes exist.	Empty state — illustration + “Tap + to create your first note”

5.2 Step 2 — Map the screen flow

Before building anything in Flutter, the screens and transitions between them were mapped so navigation was planned rather than improvised:

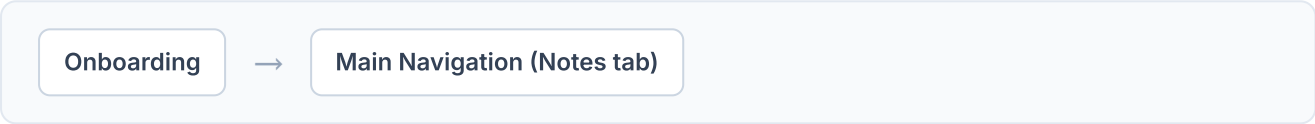


Figure 1: Path to the Notes module.

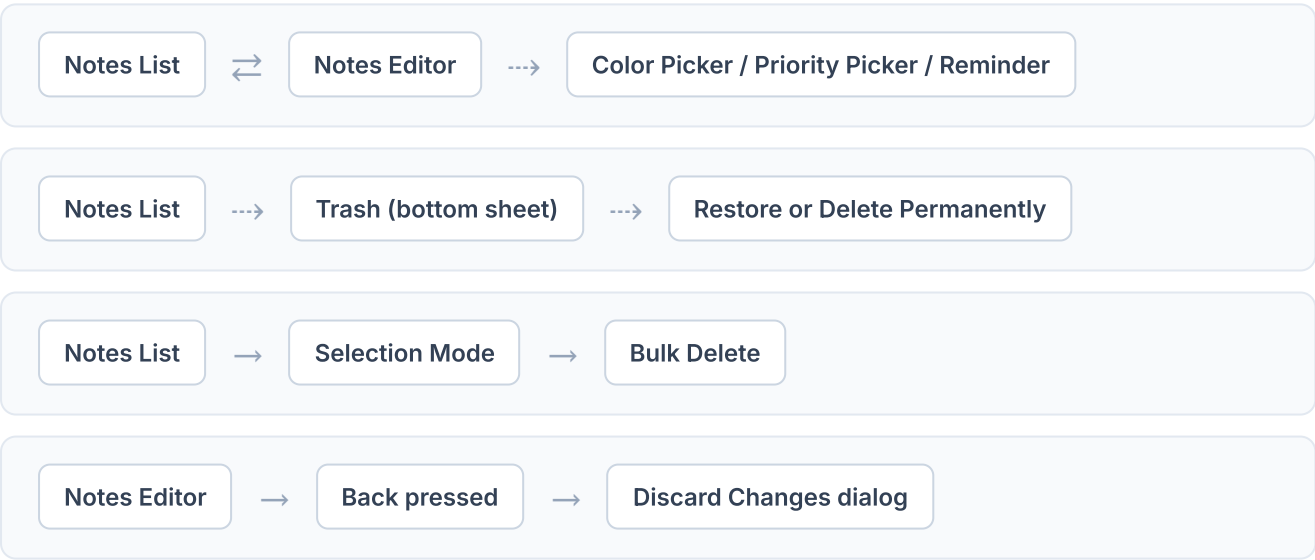


Figure 2: Notes module screen flows. The bottom navigation bar persists on every screen.

5.3 Step 3 — Build the prototype

The Notes module prototype is implemented as part of a complete Flutter application with 14 distinct UI views covering all edge cases:

- **14 unique views** — Loading, Error, Empty, List with data, Search (with results), Search (no results), Selection Mode, Editor, Color Picker Sheet, Priority Picker Sheet, Trash Sheet (with items), Trash Sheet (empty), Discard Changes dialog, Custom Color dialog.

- **Reusable components** — FAB, Bottom Sheet, Dialog, Progress Bar, Chip, Card — built once and reused across all views.
- **Persistent navigation** — A Material 3 bottom navigation bar is present on every screen with the Notes tab highlighted.
- **State coverage** — Every async operation has Loading → Error → Empty → Data states with retry and feedback.

Notes Module — View Inventory

Category	Views
List States	Loading, Error, Empty, With Data
Search	Search with results, Search no results
Actions	Selection Mode, Editor Screen
Sheets	Color Picker (14 presets + custom), Priority Picker (None/Low/Medium/High), Trash (with items), Trash (empty)
Dialogs	Delete confirmation, Discard changes, Custom Color (HSV/Hex)

6. Activity 2 — Apply and Verify Consistency

Activity 2 was implemented by fixing four rules once and reusing them across every Notes screen, rather than styling each screen independently.

6.1 Consistency rules applied

Button Placement

The create button is always a **blue (#2563EB) FAB** at bottom-right on the Notes list. In the editor, **Save** is top-right, **Delete** (trash icon) is top-right. Dialogs always place the confirm action on the right and cancel on the left.

Label Wording

'Save', 'Cancel', 'Delete', 'Discard', 'Retry', 'Apply' never change across screens. Confirmation dialogs echo the verb: 'Move to trash?', 'Discard changes?'. The editor commit button always says 'Save' (or 'Create' for new notes).

Colour Scheme

🔵 **Blue** — primary action, active tab, FAB, Apply button
🔴 **Red** — destructive (delete, discard), high priority chip
🟢 **Green** — success, low priority chip
🟠 **Amber** — warning, medium priority chip
No colour is reused for a second meaning.

Icon Position

Search is always the leftmost icon in the app bar. The create FAB is always bottom-right. Edit/tap on a note navigates to the editor. Delete/trash is always top-right of its item or card. The back arrow is always top-left in the editor.

6.2 How this builds an accurate mental model

Because the same four rules are reused rather than redesigned per screen, a user only needs to learn the pattern once. After using the Notes list for one session, the user already knows — without being told — that:

- The blue FAB creates a new note.
- Long-press on any note enters selection mode for bulk actions.
- A red button means deletion (always requires a confirmation dialog).
- A “Discard” dialog has two buttons: “Cancel” (left, outline) and “Discard” (right, red).

- Tapping back with unsaved changes triggers a “Discard changes?” confirmation.
- Blue = tappable/actionable, grey = disabled/read-only, amber = medium priority, red = high priority.

This predictability is the practical outcome this experiment asks students to demonstrate.

Consistency Matrix — Verified Across All 14 Notes Views

Rule	Where verified	Result
Primary button in same position	FAB, Save, Delete, Discard across all views	✓ Bottom-right (FAB) or top-right (editor actions)
Same action = same word	Save / Cancel / Delete / Discard / Retry / Apply	✓ Identical across all Notes screens
Each colour = one meaning	Blue (action), Red (destructive), Green (success), Amber (warning)	✓ No colour reused for a second meaning
Can user predict unseen screen?	Tested: Trash, Color Picker, Priority Picker all reuse list patterns	✓ Yes — FAB, selection, delete, discard all match
Error/Loading/Empty states	Notes list, Search, Trash	✓ Consistently handled with Retry or empty-state CTA
Destructive action protection	Delete, Discard, Permanent delete from trash	✓ Always requires confirmation dialog

---

7. Output / Result

The completed Notes module prototype is implemented within a fully functional Flutter application. The module opens with a notes list (or empty state for first-time users), provides rich-text editing via flutter_quill, and supports colour coding (14 presets + custom), priority tagging, search, selection mode for bulk actions, and a trash system with restore capability.

The design system (colour styles, text styles, and reusable widget components) is implemented directly in Dart code via shared Flutter widgets and Material 3 theme configuration, ensuring every screen follows the same placement, wording, and colour rules.

7.1 Self-evaluation checklist

✓ **Is the primary action in the same position on every screen?**

Yes — FAB bottom-right for create; Save top-right in editor; Discard right-side in dialogs

---

✓ **Does the same action always use the same word?**

Yes — Save / Cancel / Delete / Discard / Retry / Apply unchanged across all views

---

✓ **Is each colour assigned exactly one meaning?**

Yes — Blue = action, Red = destructive, Green = success/low pri, Amber = warning/med pri

---

✓ **Can a user predict an unseen screen from a seen one?**

Yes — Trash, Color Picker, and Priority Picker all reuse FAB, selection, and dialog patterns from the notes list

---

✓ **Does every loading/error state provide feedback and recovery?**

Yes — Loading spinner + Error state with Retry button

✓ **Is every destructive action protected by a confirmation dialog?**

Yes — Delete, Discard, Permanent delete all require explicit confirmation

---

✓ **Are both empty and populated states handled?**

Yes — Notes list, Search, and Trash all have empty states with CTAs

---

✓ **Does the bottom navigation persist?**

Yes — 6-tab Material 3 navigation bar with Notes tab highlighted

---

Annexure A — Selected UI/UX Ideas Integrated

Five ideas from the reference design-thinking framework were integrated into the Notes module, each mapped to a real user requirement rather than added as decoration:

1. Progressive Onboarding with Skip

Screen: Onboarding (5 pages) | **User problem it solves:** First-time users see all features (Notes, Habits, Calendar, Calculator, Life Tracker) in one flow. The Notes page introduces rich text editing before the user even opens the app.

2. Colour + Priority Visual Encoding

Screen: Notes List + Note Editor | **User problem it solves:** Users visually scan notes by colour (14 presets + custom HSV/Hex) and priority (None/Low/Medium/High with colour-coded chips). Priority colours match the app's colour semantics (Low = green, Medium = amber, High = red).

3. Search with Real-Time Feedback

Screen: Notes List | **User problem it solves:** Users filter notes instantly by typing in the search bar. Both hit and no-result states are handled, with a clear "No results found" message and suggestion to try a different term.

4. Trash with Restore Safety Net

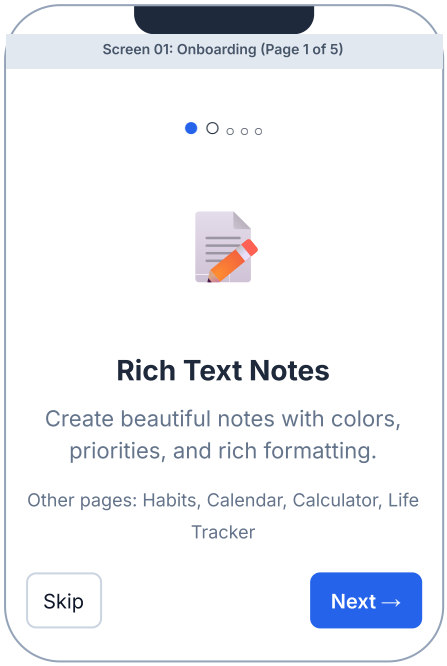
Screen: Trash Bottom Sheet | **User problem it solves:** Deleted notes are not permanently removed immediately. Users can restore notes from trash or permanently delete them. The trash clearly shows deletion timestamps and offers "Empty Trash" for bulk cleanup.

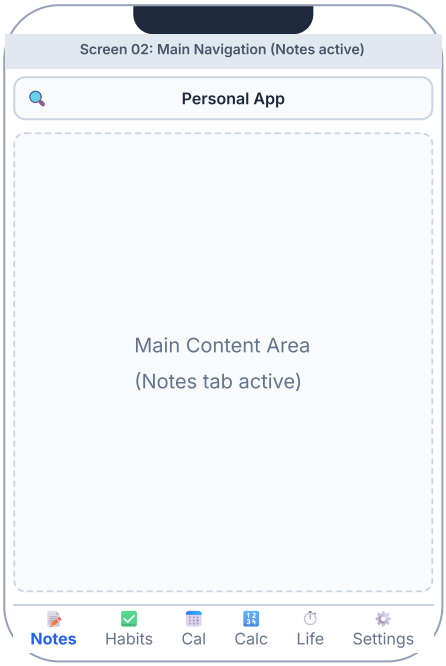
5. Discard Changes Confirmation

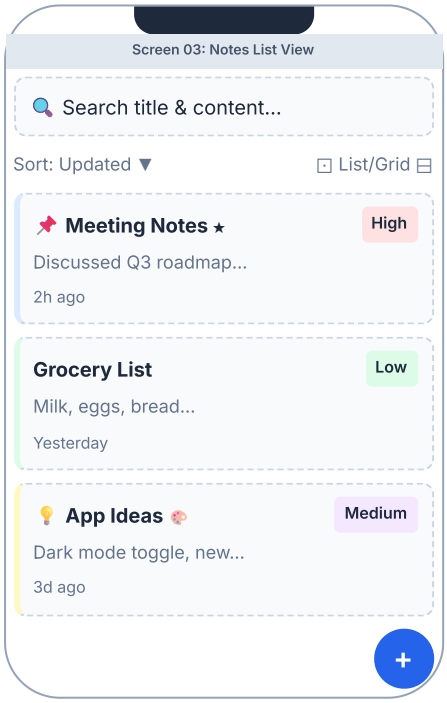
Screen: Notes Editor | **User problem it solves:** Prevents accidental data loss by showing a confirmation dialog when the user presses back with unsaved changes. The dialog uses consistent wording ("Discard changes?") and buttons ("Cancel" / "Discard").

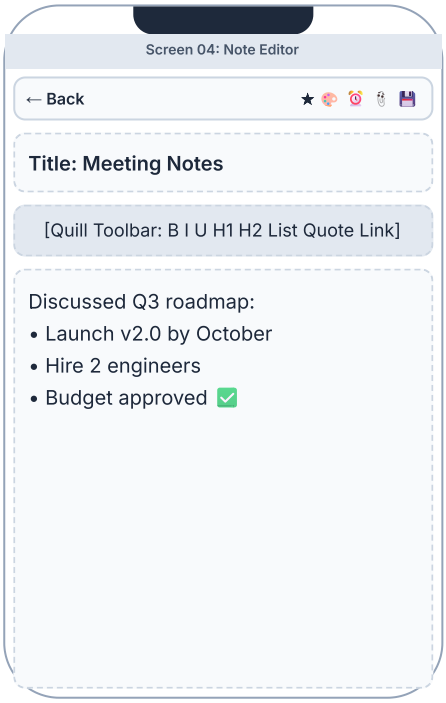
Annexure B — Screen Wireframes (Notes Module)

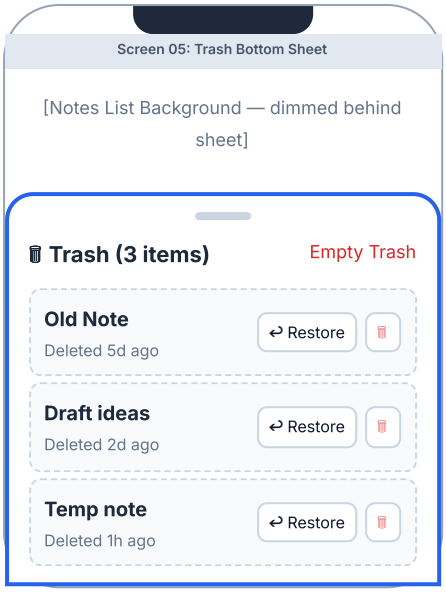
Six representative screens from the Notes module, showing the full user journey from onboarding through navigation, notes list, editor, trash, and colour picker.

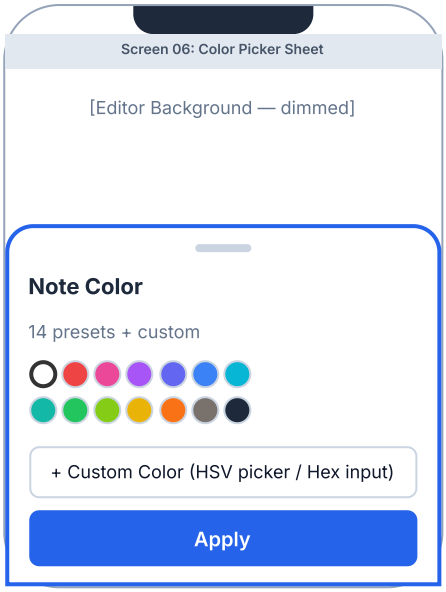









Screen 05: Trash Bottom Sheet
[Notes List Background — dimmed behind sheet]
Trash (3 items)
Empty Trash
Old Note
Deleted 5d ago
Restore
Draft ideas
Deleted 2d ago
Restore
Temp note
Deleted 1h ago
Restore


Screen 06: Color Picker Sheet
[Editor Background — dimmed]
Note Color
14 presets + custom
+ Custom Color (HSV picker / Hex input)
Apply